RESEARCH BLUEPRINT

NGSolve Agent

Reliable agentic workflows for finite element simulations

Research question

How do different LLM orchestration and interface strategies affect the reliability, numerical correctness, and reproducibility of automated finite element workflows in NGSolve?

CENTRAL RECOMMENDATION

Do not frame the project as another generic “LLM that writes FEM code”. Build an NGSolve-focused evaluation framework in which numerical verification, controlled error recovery, and reproducible experiments are first-class components. Compare D1 and D4 as the primary controlled contrast; include D3 for retrieval-grounded driver-code generation; keep D2 exploratory.

Prepared as a working technical and research specification
Version 1.0 | 7 November 2026

Executive summary

The project should investigate whether a carefully constrained agent can turn a scientific task into a reproducible NGSolve simulation workflow: formulate the problem, prepare or generate driver code, execute the solver, post-process the result, and verify that the numerical answer is credible. The key distinction is between code that runs and a simulation that is mathematically and numerically correct.

- Core contribution: an NGSolve-specific benchmark and evaluation harness that checks more than syntax or execution.
- Primary architecture comparison: D1 (LLM selects among typed tools) versus D4 (deterministic LangGraph workflow controls the sequence).
- Additional comparison: D3 (RAG-grounded generation of a driver script), because driver-code generation is part of the intended workflow.
- Optional arm: D2 (one large declarative configuration). Include it only if a clean, meaningful NGSolve schema can be designed.
- Required baseline: B0, a deterministic non-agent implementation. Without B0, the added value of an agent is difficult to quantify.
- Start with a small family of Poisson problems and one suitable TU Delft course task; expand to elasticity or Maxwell problems only after the core evaluation works.

SCOPE GUARDRAIL

The reviewed literature already contains LLM-driven FEM/FEA code generation, retrieval, multi-agent workflows, and benchmarks. Novelty therefore cannot be “an LLM uses tools to write FEM code”. The defensible target is a carefully evaluated NGSolve workflow with explicit numerical verification, controlled orchestration comparisons, and transparent failure analysis.

How to use this document

Treat this as a living project specification, not as a claim that every implementation decision has already been validated. The literature section records the state of the field found in the sources listed at the end. The architecture, task suite, metrics, and roadmap are proposed design choices that should be revised after a small technical pilot.

1. Research framing and scope

1.1 Project objective

Build and evaluate a specialized agentic system for NGSolve that accepts a well-specified FEM task, constructs a valid numerical workflow, executes it, post-processes the result, and checks the result against independent correctness criteria. The system should expose enough intermediate state to make failures diagnosable and experiments reproducible.

1.2 Main research question

RESEARCH QUESTION

How do different LLM orchestration and interface strategies affect the reliability, numerical correctness, and reproducibility of automated finite element workflows in NGSolve?

1.3 Sub-questions

1. Does deterministic orchestration (D4) reduce failure rates or improve recovery compared with an LLM-controlled typed-tool sequence (D1)?
2. How does retrieval-grounded script generation (D3) compare with tool-mediated construction when the task requires a complete NGSolve driver script?
3. How often do generated workflows pass successive gates: execution, formulation checks, numerical accuracy, boundary-condition checks, and efficiency?
4. Which error classes dominate, and can the system detect, classify, and recover from them without introducing new errors?

1.4 What is in scope

- A bounded class of stationary PDE/FEM problems with clearly defined geometry, coefficients, boundary conditions, reference data, and tolerances.
- A controlled comparison of agent architectures under the same task set, model backend, tool contracts, and retry budget.
- Independent numerical evaluation, experiment tracking, and a reproducible benchmark harness.

1.5 What is out of scope initially

- A new finite element method or a new general-purpose FEM library.
- A universal autonomous engineering system for arbitrary physics problems.
- Fine-tuning a foundation model, a multi-agent swarm, GPU acceleration, shape optimization, and large multiphysics models.
- Treating the separate discrete harmonic center mathematics project as part of this NGSolve project. It should remain a distinct research thread.

2. Related work and what it means for this project

The literature shows that agentic automation for finite element and PDE workflows is already an active research area. The practical implication is to make the research question narrower and the evaluation stronger, rather than presenting a generic agent as the novelty.

2.1 FEABench - LLMs operating real FEA software

FEABench (NeurIPS 2024) evaluates language models and agents on physics, mathematics, and engineering tasks using COMSOL Multiphysics. Its agent interacts with the software through an API, inspects outputs, and iterates. The best reported strategy generated executable API calls 88% of the time, yet none of the tested systems completely and correctly solved any benchmark problem. This is an important precedent: API-call execution is a weak proxy for end-to-end scientific correctness.

Implication: our evaluator must distinguish “the code executed” from “the PDE was formulated correctly” and “the numerical result is accurate”.

2.2 ALL-FEM - specialized agentic FEM code generation

ALL-FEM (Computer Methods in Applied Mechanics and Engineering, 2026) is a FEniCS-focused system combining a corpus of 1,000+ verified scripts, fine-tuned models, and specialized agents for PDE formulation, code generation/debugging, consistency checking, and visualization. It evaluates 39 problems across elasticity, plasticity, flow, thermofluids, fluid-structure interaction, and related areas. The paper reports 71.79% code-level success for its best fine-tuned model in a multi-agent workflow.

Implication: do not claim that multi-agent FEM code generation, retrieval, or verification is new. Differentiate through NGSolve-specific task coverage, a transparent benchmark, orchestration ablations, and numerical rather than code-only validation.

2.3 PDEAgent-Bench - multi-metric PDE solver benchmark

PDEAgent-Bench (arXiv:2605.09636, 2026) contains 645 instances across six mathematical categories and 11 PDE families. Its stated library coverage includes DOLFINx, Firedrake, and deal.II. It evaluates solver generation through staged gates: executability, numerical accuracy, and computational efficiency. The paper reports that passing rates fall when accuracy and efficiency are required in addition to runnable code.

Implication: a staged evaluator is well motivated. In the paper description reviewed for this document, NGSolve is not among the named supported libraries. This suggests a possible NGSolve-specific benchmark niche, but it does not prove that no prior NGSolve agent or benchmark exists.

2.4 ModSolAgent - retrieval and iterative verification for Abaqus

ModSolAgent (IEEE Transactions on Industrial Informatics, 2026) generates Python scripts for Abaqus. Its approach combines structured reasoning instructions, retrieval guidance, and iterative verification; the paper reports an 83.3% success rate on its selected real-world FEA tasks.

Implication: RAG plus iterative validation is already used in engineering simulation agents. D3 should be studied as a specific interface strategy for NGSolve, not described as a novel technique by itself.

2.5 What gap remains plausible?

CAREFUL NOVELTY STATEMENT

The reviewed sources establish active work on agentic FEM/FEA automation and PDE solver benchmarks. A plausible, bounded contribution is an NGSolve-focused benchmark and experimental study that compares orchestration/interface strategies while independently verifying formulation, numerical accuracy, convergence, and recovery. This is a research hypothesis to validate with a broader literature search, not a proof of being the first.

3. D1-D4: what they compare and how to use them

The D1-D4 designs originate in the supplied conference paper “Design and evaluation of MCP-server for agentic reservoir simulation workflows” (Novikov et al., European Conference on the Mathematics of Geological Reservoirs, 2026). It compares four ways to expose and orchestrate a simulator through MCP. They are useful as design patterns, but they are not four perfectly isolated variants of one architectural variable.

Design	Interaction pattern	Who controls the sequence?	Main strength	Main risk
D1 - typed tools	Expose a curated set of strict, typed tool calls; the model chooses and orders calls.	Primarily the LLM.	Local contracts; each action can be validated immediately.	Long tool-call sequences can accumulate transcription or planning errors.
D2 - declarative config	The model emits one structured configuration document.	The LLM produces the whole artifact in one step.	Compact interface when a stable schema fully describes the task.	A single global artifact can be internally inconsistent; errors may be difficult to localize.
D3 - RAG code generation	Retrieve API documentation/examples and generate a runnable driver script.	The LLM writes the script; runtime/evaluator checks it.	Flexible; close to the real driver-code-generation use case.	Retrieved examples do not guarantee valid API calls or correct numerical formulation.
D4 - deterministic workflow	A finite-state graph handles ordering, validation, retries, and recovery; the model is called at bounded nodes.	Primarily deterministic application code.	Auditable control flow, bounded decisions, predictable recovery.	Less flexible outside the workflow explicitly encoded by developers.

3.1 Important methodological nuance

D1, D2, and D3 mainly change the interface through which the model influences the simulator: tools, a structured artifact, or generated code. D4 changes the control policy by moving workflow ordering and recovery into deterministic application logic. Therefore, comparing all four as if only one independent variable changes would be methodologically weak.

BEST CONTROLLED CONTRAST

Make D1 versus D4 the primary comparison and keep the underlying typed operations as similar as possible. This makes the question more interpretable: what changes when the model controls the tool sequence versus when the application controls the sequence? Evaluate D3 separately as the script-generation arm.

3.2 What the supplied D1-D4 paper reports

- The evaluation used seven reservoir model cases, three independent repeats per design/problem/model combination, and up to three attempts per repeat.
- D4 passed all 21 runs for each of the two hosted models; all those runs passed on the first attempt. The locally hosted Gemma 4 31B model passed six of seven D4 problem cases.
- D1 and D4 had the strongest observed success. D2 had at least one failure on every problem over the three repeats. D3 failures were predominantly runtime API execution errors; the strongest hosted model solved 16 of 18 reported D3 runs.

- The paper used exact normalized configuration matching. One D4 result was physically equivalent to the reference but included a redundant extra field, so it failed the exact-match gate.

These are results for the Open-DARTS reservoir simulation tasks and the exact setup in that paper. They are not predictions of how D1-D4 will behave in NGSolve. The exact-match example is especially relevant: our evaluator should distinguish semantic/numerical equivalence from strict artifact identity.

3.3 Recommended experimental arms

Arm	Role in the study	Recommendation
B0 - deterministic baseline	Runs a hand-written, known-correct NGSolve workflow without an LLM.	Required. Establishes the reference implementation and cost floor.
D1 - typed tool agent	The LLM chooses calls to narrowly scoped FEM tools.	Primary arm.
D4 - deterministic graph	LangGraph/application controls sequence; LLM handles bounded interpretation or code decisions.	Primary arm.
D3 - RAG script generation	The model retrieves NGSolve docs/examples and writes a driver script.	Include after B0 and a basic D4/D1 system work.
D2 - structured config	The model returns one declarative task description consumed by deterministic code.	Optional/exploratory; include only if the schema is expressive but not unwieldy.

4. Benchmark: start with a small, defensible task suite

The benchmark should be deliberately narrow at first. It is better to have six well-specified problems with analytical or trusted reference solutions than a large collection of tasks whose correctness cannot be evaluated independently.

ID	Problem family	Why it belongs	Reference/check
P1	Poisson equation on the unit square with a manufactured exact solution.	Minimal end-to-end baseline; tests weak form, mesh, FE space, solver, and post-processing.	Analytical solution; relative L2/H1 error; BC check; mesh refinement.
P2	Poisson problem with mixed Dirichlet and Neumann boundary conditions.	Tests boundary tagging and correct treatment of natural boundary terms.	Analytical/manufactured solution; flux and boundary-condition checks.
P3	Variable-coefficient Poisson equation.	Tests coefficient handling and integration of spatially varying data.	Manufactured solution; reference grid; error norms.
P4	Poisson equation on a nontrivial geometry.	Tests geometry construction, boundary labels, and mesh generation.	Trusted reference solution; mesh/geometry invariants; numerical norms.
P5	Adaptive mesh refinement for an elliptic problem.	Tests a multi-stage workflow and whether error indicators/refinement improve the solution.	Error versus DOF curve; refinement history; stopping criteria.
P6	One suitable TU Delft course exercise.	Connects the project to a real educational FEM task, as suggested by the supervisor.	Use only if task statement, permitted use, and reference/solution criteria are available.

4.1 Expansion only after the baseline is stable

- Linear elasticity is a natural next PDE family once vector-valued spaces and stress/strain post-processing are reliable.

- Maxwell problems can be a later extension, with the appropriate H(curl) spaces and divergence/constraint checks.
- Navier-Stokes, fluid-structure interaction, shape optimization, and multiphysics should not be first-wave tasks. They add modeling and solver complexity before the agent/evaluator pipeline is proven.

4.2 Why not start with SuiteSparse matrices?

The SuiteSparse/Dziekonski matrices are more naturally a separate sparse linear algebra or solver/preconditioner benchmark. They do not directly test the agent's ability to translate a scientific PDE specification into geometry, function spaces, weak forms, boundary conditions, and post-processing. Keep them as a later, distinct experiment if solver selection or sparse-system performance becomes a project question.

5. Evaluation: make correctness measurable

NON-NEGOTIABLE PRINCIPLE

A runnable Python script is not proof that the intended PDE was solved. Every benchmark run must pass a sequence of independent gates.

Gate	Question	Example evidence
G0 - specification	Was the natural-language task converted into the intended mathematical problem?	PDE, coefficients, domain, boundary conditions, units, assumptions.
G1 - artifact	Is the generated code or structured input well-formed?	Schema validation, import checks, API/signature checks, static checks.
G2 - execution	Does the workflow run to completion?	Exit status, solver status, exceptions, timeout.
G3 - formulation	Does the discrete formulation represent the specified problem?	Weak-form structure, FE space compatibility, boundary labels and terms.
G4 - numerical correctness	Is the computed solution accurate enough?	Analytical/reference comparison, L2/H1 error, residual, boundary checks.
G5 - convergence/efficiency	Does refinement behave sensibly and at acceptable cost?	Error-versus-DOF trend, runtime, DOFs, iterations, memory if measured.
G6 - reproducibility	Can the run be reconstructed and audited?	Versions, prompts, configuration, seed, generated code, logs, raw metrics.

5.1 Core metrics

Metric	Operational definition	Why it matters
End-to-end success rate	Fraction of runs passing all required gates.	Primary system-level reliability metric.
Execution rate	Fraction of runs that execute without fatal errors.	Separates software failures from scientific failures.
Formulation correctness	Fraction matching the specified PDE, domain, spaces, and BCs.	Catches a solver that runs on the wrong problem.
Relative L2 error	||u_h - u_ref||_L2 / ||u_ref||_L2 (when a reference is defined).	Measures solution error in an interpretable norm.
Relative H1 error	Corresponding H1 norm error where appropriate.	Includes derivative/gradient quality.
Residual and BC violation	Norms of discrete/continuous residual proxies and boundary-condition mismatch.	Checks equations and constraints not captured by script validity.
Convergence trend	Error versus mesh size or degrees of freedom.	A single mesh can hide non-convergent or inconsistent behavior.
Recovery rate	Fraction of injected/observed failures correctly	Measures whether iteration actually improves

Metric	Operational definition	Why it matters
	diagnosed and repaired.	reliability.
Cost and latency	Tokens, model calls, wall time, solver time, retries.	Quantifies practical overhead.
Human intervention	Number and type of manual corrections or approvals.	Shows whether the workflow is truly autonomous or supervised.

5.2 Reference solutions and tolerances

For initial Poisson tasks, use manufactured solutions wherever possible. Choose an exact smooth function u(x,y), derive the forcing term and boundary data consistently, and compare the computed finite-element solution against that exact function. For geometries or tasks without closed-form solutions, use a trusted reference solution produced by a separately validated implementation and evaluate on a prescribed grid or with a clearly specified norm.

Do not choose numerical tolerances after seeing model outputs. Define them before the experiment, justify them by mesh order/problem conditioning, and record them in the benchmark specification. Keep both strict checks and continuous error values so results are not reduced to a single pass/fail label.

6. Fault injection and recovery tests

A system that succeeds only on clean prompts is not yet a robust engineering agent. Once the baseline task suite is stable, introduce controlled faults to test whether the agent can detect, classify, and repair them.

Injected fault	What it probes	Possible evaluator signal
Wrong boundary label	Boundary-tag understanding and geometry/mesh consistency.	BC mismatch, boundary integral inconsistency, exact solution error.
Missing boundary condition	Whether the system detects an under-specified or ill-posed problem.	Specification check, singular system, failed solvability/BC gate.
Incompatible FE space	Whether the system understands the mathematical space required by the formulation.	Formulation validator or assembly error.
Incorrect weak-form sign/term	Whether the system verifies mathematics rather than syntax.	Reference error, residual, conservation/energy checks where applicable.
Inappropriate mesh parameter	Whether the workflow catches mesh-quality or resolution problems.	Mesh-quality thresholds, convergence behavior, numerical error.

6.1 Recovery protocol

1. Record the original error and the gate that detected it.
2. Return a concise diagnostic that identifies the failed invariant, not a hidden reference answer.
3. Allow a bounded correction attempt and record the exact change.
4. Re-run all relevant gates, not only the gate that failed.
5. Count a recovery as successful only if the corrected run passes the full required evaluation and does not break previously valid properties.

Also test abstention: when the user specification is genuinely ambiguous or asks for a task outside the supported problem family, the system should ask for clarification or state that the task is unsupported rather than inventing a model.

7. Experimental protocol

7.1 Keep the comparison fair

- Use the same benchmark task definitions and reference data across all architecture arms.
- Pin the model provider/model ID and version where available; record date, parameters, temperature, and reasoning settings when exposed.
- Keep tool contracts, numerical tolerances, retry budget, timeouts, and evaluator code fixed across arms wherever the design permits.
- Version prompts and retrieval corpora. D3 must use a leave-one-out or equivalent anti-leakage rule so the exact target solution is not retrieved.
- Store every run, including failures. Do not report only the best run or only successful runs.

7.2 Pilot size

A practical pilot is five to six tasks by three independent repeats per architecture. This is suitable for debugging the benchmark and identifying major failure patterns, not for strong statistical claims. After the harness is stable, increase the number of repeats and/or tasks according to compute and model budget; report uncertainty and avoid over-interpreting small differences.

7.3 Primary comparison and ablations

- Primary: B0 versus D1 versus D4.
- Secondary: D3 versus D4 on tasks that require a driver script or allow a meaningful script-generation workflow.
- Optional: D2 only when the declarative schema is a fair representation of NGSolve tasks.
- Recovery ablation: compare bounded retries with and without structured diagnostic feedback.
- Retrieval ablation: compare D3 with and without retrieval, while keeping the task examples leakage-free.

7.4 Statistical reporting

For binary outcomes, report counts and denominators alongside percentages (for example, 13/18, not only 72%). For numerical error and runtime, report median and spread, and include per-task results. With a small pilot, describe patterns and failure cases rather than claiming broad statistical superiority.

8. Proposed software architecture

Separate the deterministic FEM engine, the agent/orchestration layer, the independent validator, and the experiment harness. The evaluator should not simply trust the agent's self-reported success.

Layer	Responsibility	Implementation direction
Task specification	Normalize problem statement into typed fields: PDE, domain, coefficients, BCs, FE family/order, solver and output requirements.	Pydantic models; explicit assumptions and validation errors.
FEM core	Construct geometry/mesh, spaces, forms, solve, and post-process.	Small tested NGSolve functions; no LLM logic inside core numerical routines.
Tool interface	Expose safe, narrow operations to D1 and bounded nodes in D4.	Typed Python functions first; MCP wrapper later.
Agent/workflow	Interpret the task, select allowed actions, route and recover.	LangGraph for D4; a tool-calling loop for D1.
Retrieval	Retrieve NGSolve documentation and vetted examples for D3.	Versioned local corpus; metadata, source version, and task-family tags.
Execution runner	Run generated scripts in a controlled environment.	Dedicated worker, isolated workspace, timeouts, resource limits, limited filesystem/network access.
Validation	Independently check syntax, formulation, accuracy, convergence and efficiency.	Separate modules; known reference data and benchmark-specific checks.
Experiment/reporting	Record provenance, metrics, plots and per-run	JSON/CSV raw records, plots, summary

Layer	Responsibility	Implementation direction
	report.	tables.

8.1 MCP: introduce it after the core works

MCP is useful as a standardized external interface, but it does not need to be the first thing implemented. First make the NGSolve operations work as ordinary Python functions and test them. Then wrap those functions as MCP tools. This separates FEM correctness from protocol and transport issues.

8.2 Safe execution of generated code

EXECUTION BOUNDARY

Do not execute arbitrary model-generated Python with unrestricted process access. A timeout alone is not a sandbox. Use a dedicated runner with a separate working directory, controlled inputs/outputs, process limits, restricted filesystem and network access, and a disposable environment. Begin with fixed templates and constrained parameters before allowing more general scripts.

9. Coding environment and setup

9.1 Recommended starting environment

Component	Choice	Reason / note
Operating system	Windows 11 native	Start with the simplest path that supports the official NGSolve wheel.
Python	Python 3.12, 64-bit	A current compatible interpreter with published NGSolve Windows wheels; pin the exact tested combination.
Editor	Visual Studio Code	Python extension, terminal, testing, Git integration.
Environment/package manager	uv	Project metadata, virtual environment, dependency resolution and lockfile.
FEM engine	NGSolve	Core finite-element solver and mesh ecosystem.
Agent framework	LangGraph	Stateful workflows with deterministic routing and bounded LLM nodes.
Validation/data models	Pydantic v2	Typed task schemas and explicit validation.
Testing/style	pytest and ruff	Automated correctness tests and consistent Python style.
Numerical/data tools	NumPy, SciPy, Matplotlib	Reference calculations, arrays, plots and diagnostics.
Protocol	MCP Python SDK	Add only after ordinary tool functions are stable; SDK v2 docs require Python 3.11+.

The official NGSolve installation guide recommends pip installation for supported platforms. PyPI currently lists Windows x86-64 wheels for CPython 3.12, so begin with a normal wheel installation rather than compiling NGSolve from source. The exact latest package version can change; let uv resolve it, test it, and preserve the tested version in uv.lock.

9.2 Initial PowerShell setup

```
uv python install 3.12
uv init ngsolve-agent
cd ngsolve-agent
uv venv --python 3.12
.venv\Scripts\Activate.ps1
uv add ngsolve numpy scipy matplotlib pydantic langgraph
uv add --dev pytest ruff
```

After installation, create a minimal NGSolve Poisson example and verify that it imports, assembles, solves, and writes a result. Only then add agent-provider packages and MCP. Keep uv.lock under version control so the project can be recreated.

9.3 Dependency and environment rule

If the FEM solver and agent stack later develop incompatible dependency constraints, split them into two environments with a narrow interface: an NGSolve worker process and an agent/orchestrator process. Do not introduce this complexity before there is evidence that it is needed.

9.4 MCP SDK versioning

The current official MCP Python SDK documentation describes the v2 line and notes breaking changes from v1. Do not mix v1 FastMCP snippets with v2 examples. Select one SDK line, pin it, and test the exact server/client API used by the project.

10. Suggested repository structure

```
ngsolve-agent/
|-- README.md  pyproject.toml  uv.lock
|-- src/ngsolve_agent/
|   |-- fem/{geometry, mesh, spaces, forms, solver, postprocessing}.py
|   |-- agent/{state, nodes, prompts, workflow}.py
|   |-- validation/{schemas, semantic, formulation, numerical}.py
|   |-- retrieval/{ingestion, indexing, search}.py
|   |-- execution/{runner, sandbox}.py
|   |-- benchmark/{cases, evaluator, metrics}.py
|   `-- reporting/{plots, reports}.py
|-- benchmarks/{problems, reference_solutions, results}/
|-- examples/poisson/
|-- tests/{test_fem, test_validation, test_numerical, test_execution, test_workflow}.py
|-- experiments/{configs, run_experiment.py}
`-- results/{raw, figures, reports}/

Grouped names in braces are a compact notation for separate files or folders.
```

10.1 What every run must record

- Experiment ID, architecture arm, task ID, repeat number, timestamp, model/provider/version, prompt version or hash.
- Python, NGSolve, dependency and operating-system versions; mesh parameters, FE family/order, solver parameters, tolerances.
- Generated code/configuration, tool-call trace, execution result, gate outcomes, numerical error, runtime, tokens, retries and human intervention.
- Reference solution/version, evaluator version, random seed if applicable, and paths/hashes for raw artifacts.

11. Implementation roadmap

Stage	Build	Exit criterion
0 - define scope	Freeze the initial PDE family, task definitions, exact/reference solutions, and evaluation tolerances.	Each initial task has an unambiguous mathematical specification and independent expected checks.
1 - FEM baseline	Implement P1-P5 manually in NGSolve, with tests and plots.	Reference runs are repeatable; errors and convergence are measured.

Stage	Build	Exit criterion
2 - typed task schema	Represent domain, PDE, coefficients, BCs, FE choices and output requirements with Pydantic.	Invalid/incomplete inputs are rejected before solver execution.
3 - deterministic tools	Create small NGSolve functions for geometry, mesh, spaces, forms, solve and post-processing.	Tools are individually tested and their side effects are controlled.
4 - B0 and evaluator	Run the hand-written baseline through the same evaluator as agent systems.	Every task emits a machine-readable report with all gates and metrics.
5 - D4 workflow	Implement deterministic routing, bounded LLM calls, retries and explicit state.	Workflow trace is auditable; unsupported/ambiguous requests are handled explicitly.
6 - D1 tool agent	Allow an LLM to select among the same narrowly scoped typed tools.	Controlled D1-D4 comparison can be run on identical tasks.
7 - D3 retrieval/codegen	Index vetted NGSolve documentation/examples and generate bounded driver scripts.	No target solution leakage; generated code runs in an isolated runner.
8 - benchmark study	Run repeats, collect faults, analyze metrics and classify failures.	Results include all runs, uncertainty, per-task breakdowns and limitations.
9 - expand scope	Add elasticity or Maxwell tasks and possibly a separate sparse-solver benchmark.	The original task family remains stable and reproducible.

12. What to add now, and what to postpone

Priority	Feature	Decision
Must have	Independent numerical verification	Core scientific contribution; implement before agent complexity.
Must have	Benchmark harness and experiment logging	Required for reproducibility and fair comparisons.
Must have	Failure taxonomy and bounded recovery	Measure how the system fails, not only success rate.
Should have	RAG over NGSolve documentation	Needed for D3; build from official docs and vetted examples.
Should have	Human-in-the-loop / abstention	Useful for ambiguous, unsupported, or high-cost tasks.
Later	MCP server/client wrapper	Add after local Python operations and workflow are stable.
Later	SuiteSparse matrix experiments	Separate solver/preconditioner track, not first benchmark core.
Defer	Shape optimization and complex multiphysics	Too much modeling/solver complexity for the first study.
Defer	Multi-agent swarm and fine-tuning	Do not add before a single-agent baseline demonstrates a measurable bottleneck.
Defer	GPU acceleration	Not central to the initial reliability research question.

13. Main risks and how to avoid them

Risk	Why it matters	Mitigation
Overclaiming novelty	Related FEM agents and PDE benchmarks already exist.	State a narrow NGSolve-specific contribution; do a broader systematic literature search before claiming priority.
Code-level scoring only	A valid script may solve the wrong PDE.	Use independent formulation and numerical gates.
Architecture confounding	D1-D4 do not change one common variable.	Prioritize D1 vs D4 with shared typed operations; treat D3/D2 as additional interface arms.
Reference leakage	RAG could retrieve the exact target script.	Use leave-one-out retrieval and log retrieved documents.
Exact-match false negatives	Semantically equivalent outputs may be rejected by string/config equality.	Use schema checks plus semantic/numerical equivalence checks; report strict identity separately.
Tiny sample size	A few repeats cannot support broad generalizations.	Report denominators and uncertainty; expand repeats after the pilot.
Unsafe arbitrary code execution	Generated scripts can modify files or consume resources.	Constrained templates and isolated execution runner with least privilege.
Overbuilding too early	Many frameworks obscure whether the FEM workflow itself works.	Start with hand-written NGSolve baselines and add components one at a time.

14. Final recommended direction

PROJECT POSITIONING

NGSolve Agent: An evaluation framework for reliable agentic finite element workflows, combining typed FEM operations, deterministic orchestration, retrieval-grounded code generation, and independent numerical verification.

The strongest next move is not to implement all four D1-D4 designs immediately. First make one small NGSolve Poisson workflow work end to end without an LLM. Turn it into a reproducible reference case. Then build the evaluator and D4 workflow, followed by D1. Add D3 when the retrieval corpus and safe script runner exist. Treat D2 as optional until a useful NGSolve configuration schema is clear.

That sequence preserves the important research question while reducing implementation risk. It also gives the project a credible scientific center: whether the agent reliably produces a correct numerical solution, not merely whether it can produce Python code.

References and source notes

The following sources support the literature review and technical setup. Links were checked on 7 November 2026. For the D1-D4 analysis, the supplied conference manuscript is treated as a user-provided source, not as an independently peer-reviewed publication.

[1] Brenner, M. et al. “FEABench: Evaluating language models on real world physics reasoning ability.” NeurIPS 2024. https://research.google/pubs/feabench-evaluating-language-models-on-real-world-physics-reasoning-ability/

[2] Deotale, R. et al. “ALL-FEM: Agentic Large Language Models fine-tuned for finite element methods.” Computer Methods in Applied Mechanics and Engineering, 457, 118985 (2026). DOI: 10.1016/j.cma.2026.118985. https://doi.org/10.1016/j.cma.2026.118985

[3] Hang, Z. et al. “PDEAgent-Bench: A Multi-Metric, Multi-Library Benchmark for PDE Solver Generation.” arXiv:2605.09636 (2026). https://arxiv.org/abs/2605.09636

[4] “ModSolAgent: Automated Finite Element Code Generation for Abaqus via LLM-Based Agent.” IEEE Transactions on Industrial Informatics, 22(6), 4693-4704 (2026). DOI: 10.1109/TII.2026.3669495. https://doi.org/10.1109/TII.2026.3669495

[5] Novikov, A., Serrao Seabra, G., Agrawal, N., Garcia Alvarez, M., Voskov, D. “Design and evaluation of MCP-server for agentic reservoir simulation workflows.” European Conference on the Mathematics of Geological Reservoirs, Porto, 8-11 September 2026. Source: supplied PDF, 158.pdf.

[6] Netgen/NGSolve documentation. “Installation.” Official installation guide. https://ngsolve.org/installation.html

[7] NGSolve package metadata and Windows wheels. PyPI project page. https://pypi.org/project/ngsolve/

[8] LangGraph documentation. “Install LangGraph.” https://docs.langchain.com/oss/python/langgraph/install

[9] Model Context Protocol. “MCP Python SDK - Installation” (v2 documentation). https://py.sdk.modelcontextprotocol.io/get-started/installation/

Important limitations of the evidence

- Published benchmark results are specific to their datasets, models, software libraries, and evaluation gates. They should not be transferred quantitatively to NGSolve without replication.
- The current literature scan is targeted, not exhaustive. It is enough to identify nearby work and shape a cautious research question, but not enough to establish a definitive “first”.
- The NGSolve setup is a recommended starting point. Package compatibility and API details must be confirmed by a clean installation and a minimal solver test in the project environment.